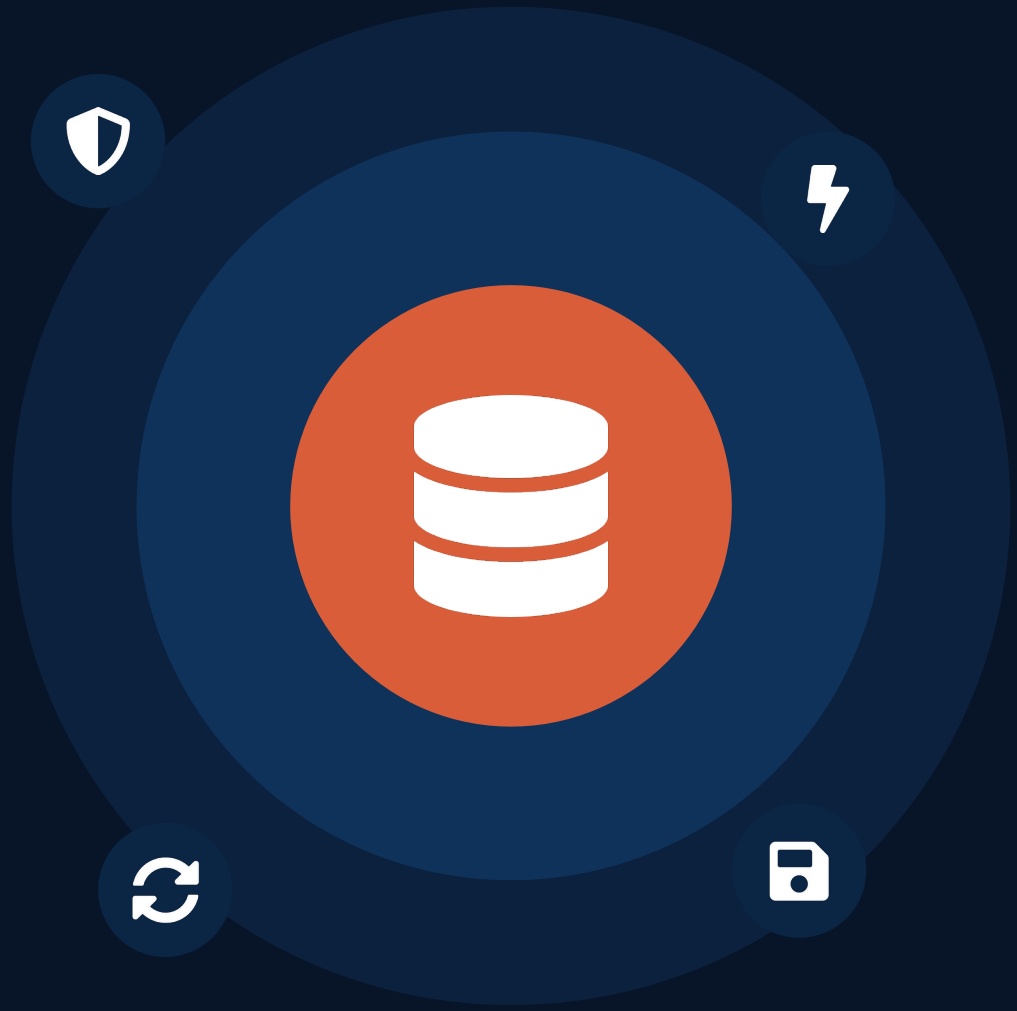
CURSO DE FORMACIÓN · 20 H

Administración de SQL Server

De la arquitectura del motor a la alta disponibilidad: operar bases de datos en producción



BIENVENIDA

Qué vamos a aprender: 20 horas, 4 módulos

0,5 h
M0 · Bienvenida y entorno

4,0 h
M1 · Fundamentos y arquitectura

5,5 h
M2 · Gestión y seguridad

5,0 h
M3 · Optimización y alta disponibilidad

5,0 h
M4 · Mantenimiento y buenas prácticas

Al terminar sabrás…

- Explicar cómo ejecuta el motor una consulta
- Crear objetos y un modelo de permisos seguro
- Leer un plan de ejecución y diagnosticar con DMVs

…y serás capaz de operar

- Diseñar backups según RPO/RTO y restaurar a un punto exacto
- Automatizar mantenimiento y resolver incidencias comunes
- Elegir la solución de HA/DR adecuada

BIENVENIDA

Cómo trabajaremos

Teoría breve

Bloques de 20-30 min con esquemas, demostraciones en vivo y fragmentos T-SQL reales.

Laboratorios guiados

Cuatro laboratorios y un caso integrador. Cada alumno trabaja en su propia instancia.

Participación

Preguntas y debates en cada tema. Los errores son material de aprendizaje.

Regla de seguridad: los scripts del curso son destructivos a propósito. Comprueba siempre SELECT @@SERVERNAME antes de ejecutarlos y nunca los lances contra producción.

Material: diapositivas con guion completo en las notas, scripts T-SQL del curso y la Guía del Docente en Markdown.

ENTORNO

Entorno de trabajo del curso

SSMS (Windows)

Cliente completo: objetos, planes de ejecución, Agent, Profiler.

VS Code + extensión MSSQL

Multiplataforma; sustituye a Azure Data Studio (fin de soporte 28/02/2026).

Docker o instalación local

Imagen oficial 2022-latest, o Developer Edition en Windows.

BASH · DOCKER

```bash
docker run -d --name sql2022 \
  -e "ACCEPT_EULA=Y" \
  -e "MSSQL_SA_PASSWORD=Curso#SQL2022!" \
  -p 1433:1433 \
  -v sqldata:/var/opt/mssql \
  mcr.microsoft.com/mssql/server:2022-latest

docker ps
docker logs sql2022 | tail -n 5
```

Si el contenedor se para: revisa docker logs. Casi siempre es una contraseña de sa que no cumple la política de complejidad.

01

MÓDULO 1 · ≈ 4,0 H

Fundamentos y arquitectura

Incluye licenciamiento: cómo es SQL Server por dentro y qué edición y licencia necesitas

- Historia y versiones
- Arquitectura del motor
- Páginas, extensiones y ficheros
- Instancias y bases del sistema
- Ediciones y licenciamiento
- Laboratorio 1

HISTORIA

Versiones de SQL Server

2012

- Always On AG
- Columnstore
- Secuencias

2014

- In-Memory OLTP
- Buffer Pool Ext.
- Backup cifrado

2016

- Query Store
- Always Encrypted
- Temporal tables

2017

- Linux y Docker
- Adaptive QP
- Python

2019

- Intelligent QP
- ADR
- UTF-8

2022

- Ledger
- PSP optimization
- Contained AG

2025

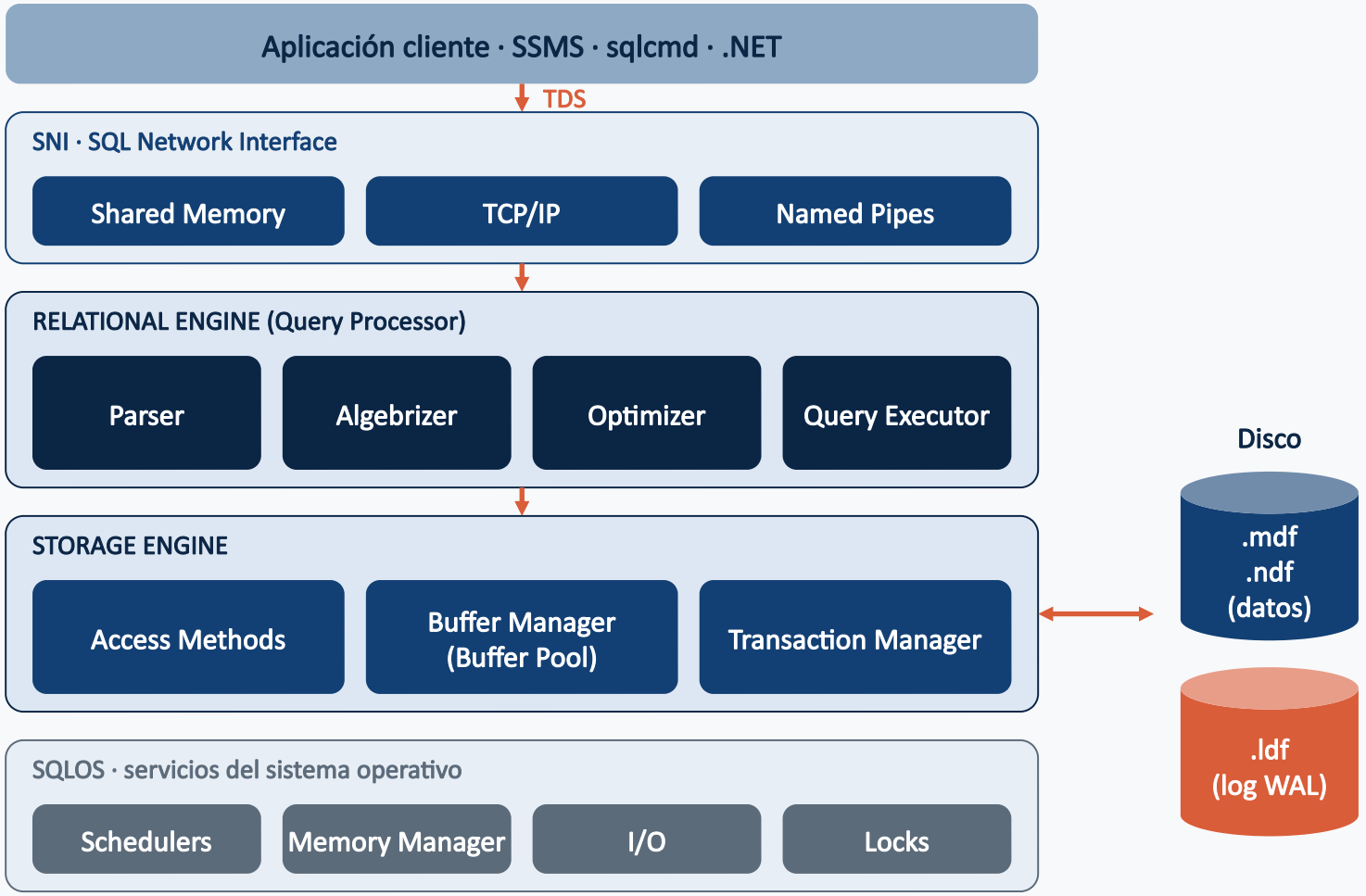
- Tipo vector (IA)
- Tipo json
- Regex en T-SQL



Ciclo de vida: 5 años de soporte principal + 5 de soporte extendido. SQL Server 2016 ya no recibe parches de seguridad (jul-2026); 2022 es la referencia del curso.

ARQUITECTURA

Arquitectura interna del motor



Relational Engine

- Decide el cómo (plan)
- Parser → Algebrizer → Optimizer
- Cost-based con estadísticas

Storage Engine

- Ejecuta el acceso a los datos
- Buffer Pool y Access Methods
- Log, bloqueos y transacciones

ARQUITECTURA

Ciclo de vida de una consulta

1

Parse + Algebrize

Sintaxis, resolución de objetos y permisos.

2

Plan cache

¿Existe un plan reutilizable? Si sí, se salta la optimización.

3

Optimize

Elige el plan de menor coste estimado con estadísticas.

4

Execute

Operadores en iteradores piden filas y páginas.

5

Buffer Pool + TDS

Lectura lógica o física; resultado al cliente.

Pruébalo: SET STATISTICS IO, TIME ON; muestra lecturas lógicas, físicas y tiempo de compilación frente a ejecución.

ARQUITECTURA

Buffer Pool y gestión de memoria

Buffer Pool

Caché de páginas de datos de 8 KB. Lectura lógica (memoria) frente a física (disco).

Plan Cache

Planes compilados reutilizables. Se pierde al reiniciar o por presión de memoria.

Checkpoint y Lazy Writer

Checkpoint escribe páginas sucias periódicamente; Lazy Writer libera memoria bajo presión.

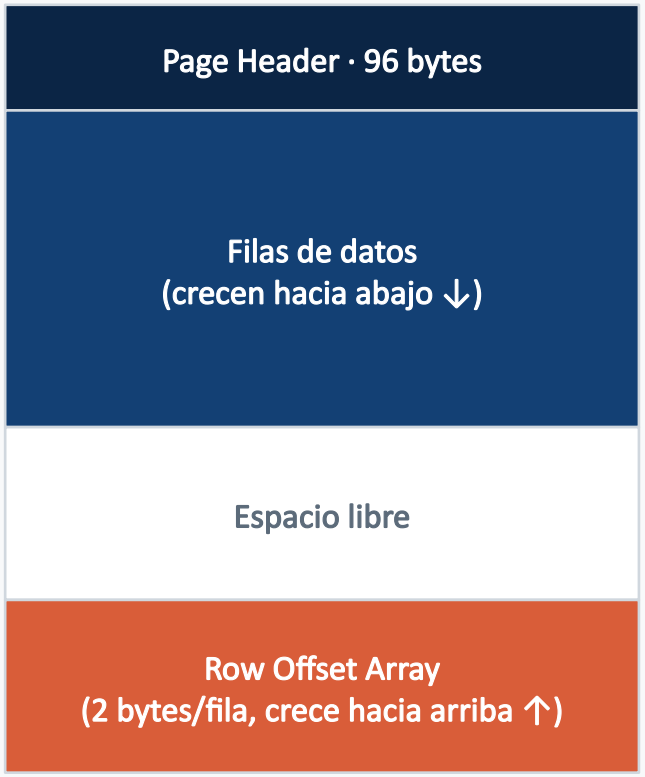
T-SQL · LIMITAR LA MEMORIA

```
EXEC sp_configure 'show advanced options', 1; RECONFIGURE;
EXEC sp_configure 'max server memory (MB)', 12288; RECONFIGURE;
```

Siempre limita max server memory en producción y deja memoria libre para el sistema operativo.

Estructura física: páginas y extensiones

Página = 8 KB (unidad mínima de E/S)



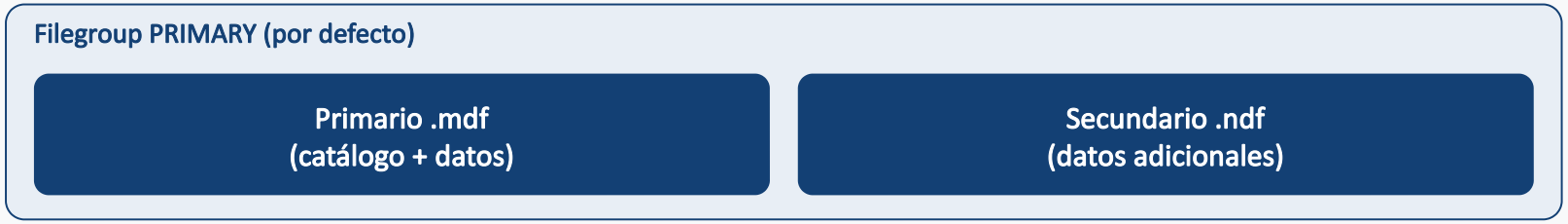
Máx. 8060 bytes de datos por fila en página (límite en fila)

Extensión = 8 páginas contiguas = 64 KB

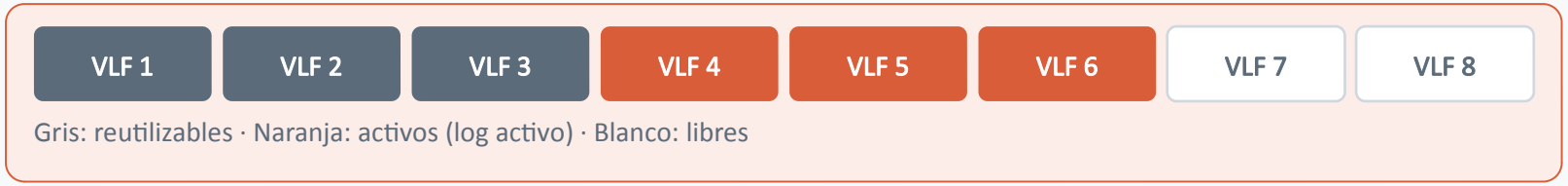


Mixta: páginas de distintos objetos · Uniforme: un solo objeto (desde la 9.ª página)

Ficheros de datos agrupados en filegroups



Transaction log (.ldf) — secuencia circular de VLFs



Gris: reutilizables · Naranja: activos (log activo) · Blanco: libres

ALMACENAMIENTO

Ficheros de datos y de log: MDF, NDF, LDF

Primario · .mdf

Uno por BD. Contiene el catálogo y, si no hay más, los datos. Filegroup PRIMARY.

Secundarios · .ndf

Opcionales. Reparten datos en varios discos y permiten restaurar por filegroup.

Log · .ldf

Registro secuencial de cambios (WAL). Con uno es suficiente y no pertenece a ningún filegroup.

Autogrowth en MB, no en %: un 10% de 500 GB son 50 GB de golpe. Dimensiona el tamaño inicial para evitar crecer.

IFI (Instant File Initialization): concede "Perform volume maintenance tasks" a la cuenta del servicio para crear y restaurar ficheros de datos mucho más rápido.

ALMACENAMIENTO

Transaction Log: WAL y VLFs

1

Se modifica la página

Cambio en el Buffer Pool; la página queda sucia.

2

Se escribe el log

Registro con LSN en el log buffer.

3

COMMIT

Flush del log a disco. El cliente recibe OK.

4

Checkpoint

Más tarde, las páginas sucias se escriben al .mdf.

T-SQL · VLFS DE LA BD ACTUAL

```
SELECT COUNT(*) AS vlf_total,
       SUM(CASE WHEN vlf_active = 1 THEN 1 ELSE 0 END) AS vlf_activos
FROM sys.dm_db_log_info(DB_ID());
```

Miles de VLFs por autogrowths pequeños ralentizan arranque y recuperación. Dimensiona el log de una vez.

Instancias: predeterminada y con nombre

Predeterminada

Nombre de servicio MSSQLSERVER. Se conecta por nombre de host, puerto TCP 1433.

Con nombre

SERVIDOR\INSTANCIA. Puerto dinámico resuelto por SQL Server Browser (UDP 1434).

T-SQL · IDENTIFICAR LA INSTANCIA

```
SELECT @@SERVERNAME                       AS servidor,
       SERVERPROPERTY('InstanceName')     AS instancia,
       SERVERPROPERTY('Edition')          AS edicion,
       SERVERPROPERTY('ProductVersion') AS version;
```

En producción: fija puertos estáticos, usa cuentas de servicio dedicadas (gMSA) y configura max server memory en cada instancia.

Bases de datos del sistema

master

Catálogo de la instancia: logins, configuración, ubicación de las BD. Sin ella no arranca.

model

Plantilla de las nuevas BD y de tempdb en cada arranque.

msdb

Agent: jobs y alertas; historial de backups; Database Mail.

tempdb

Objetos temporales, spills y version store. Se recrea al arrancar. Sin backup.

mssqlsystemresource

Sólo lectura y oculta. Contiene los objetos del sistema (sys.*).

Backup obligatorio

master, model y msdb. tempdb y Resource no se respaldan.

LICENCIAMIENTO

Ediciones de SQL Server 2022

Edición	Uso típico	Cómputo	Memoria (Buffer Pool)	Claves
Express	Apps pequeñas, aprendizaje	1 socket o 4 cores	1,4 GB	Gratuita · 10 GB/BD · sin Agent
Standard	Cargas departamentales	4 sockets o 24 cores	128 GB	Basic AG · backup comprimido
Enterprise	Misión crítica y alta carga	Máximo del SO	Máximo del SO	AG completos · online · particionado
Developer	Desarrollo y laboratorio	Como Enterprise	Como Enterprise	Gratis · prohibido en producción

Regla práctica: empieza por Standard y justifica Enterprise con una función concreta. Developer para pruebas, nunca para producción.

LICENCIAMIENTO

Modelos de licencia: Core vs Server + CAL

Per Core (por núcleo)

- Standard y Enterprise
- Usuarios y dispositivos ilimitados
- Mínimo 4 núcleos por procesador y VM
- Paquetes de 2 núcleos

Server + CAL

- Sólo Standard
- Licencia de servidor + CAL por usuario o dispositivo
- Conviene con pocos usuarios identificados
- No válido para acceso masivo desde internet

4

núcleos mínimos por procesador

2

núcleos por paquete de licencia

SA

Software Assurance: derechos de failover

HERRAMIENTAS

Herramientas de administración

SSMS

Referencia en Windows: objetos, planes gráficos, Agent, asistentes de backup/restore.

VS Code + extensión MSSQL

Multiplataforma, notebooks y Git. Sustituye a Azure Data Studio.

sqlcmd

Línea de comandos para scripts, CI y automatización.

Extended Events

Trazas ligeras. Sustituyen a SQL Server Profiler (obsoleto).

HERRAMIENTAS

sqlcmd y Extended Events básicos

BASH · SQLCMD

```
sqlcmd -S localhost,1433 -U sa -C \
  -P "Curso#SQL2022!" \
  -Q "SELECT @@VERSION"

sqlcmd -S .\SQLEXPRESS -E -b \
  -i script.sql -o salida.txt
```

T-SQL · EXTENDED EVENTS

```
CREATE EVENT SESSION [lentas] ON SERVER
ADD EVENT sqlserver.sql_statement_completed
  (WHERE duration > 1000000)   -- > 1 s (µs)
ADD TARGET package0.event_file
  (SET filename = N'lentas.xel')
WITH (STARTUP_STATE = OFF);
GO
ALTER EVENT SESSION [lentas] ON SERVER
  STATE = START;
```



system_health ya está activa por defecto y recoge deadlocks y errores graves. Mírala primero tras un incidente.



Filtra siempre con predicados para limitar la sobrecarga.

Lab 1 (I): BD con ficheros y filegroups

T-SQL · LABORATORIO 1

```
CREATE DATABASE Ventas
ON PRIMARY
  (NAME = Ventas_sys, FILENAME = '/var/opt/mssql/data/Ventas.mdf',
   SIZE = 64MB,  FILEGROWTH = 64MB),
FILEGROUP FG_DATOS
  (NAME = Ventas_d1,  FILENAME = '/var/opt/mssql/data/Ventas_d1.ndf',
   SIZE = 256MB, FILEGROWTH = 128MB),
  (NAME = Ventas_d2,  FILENAME = '/var/opt/mssql/data/Ventas_d2.ndf',
   SIZE = 256MB, FILEGROWTH = 128MB)
LOG ON
  (NAME = Ventas_log, FILENAME = '/var/opt/mssql/log/Ventas.ldf',
   SIZE = 128MB, FILEGROWTH = 64MB);
GO
```



Enunciado

- Crea la BD Ventas con los ficheros indicados
- Dos ficheros del mismo tamaño en FG_DATOS
- Autogrowth en MB, nunca en %
- Comprueba el resultado en SSMS (Propiedades → Archivos)

LABORATORIO · BASE DE DATOS

Lab 1 (II): verificar y hacer crecer

T-SQL · VERIFICACIÓN

```
ALTER DATABASE Ventas MODIFY FILEGROUP FG_DATOS DEFAULT;
GO
SELECT name, physical_name, size*8/1024 AS size_mb,
       growth*8/1024 AS growth_mb, is_percent_growth
FROM Ventas.sys.database_files;

SELECT name, is_default FROM Ventas.sys.filegroups;
```

Comprueba

Cuatro ficheros, dos filegroups, FG_DATOS predeterminado.

Vigila

is_percent_growth = 0 en todos los ficheros.

Cierre del módulo: Buffer Pool, páginas y extensiones, Transaction Log (WAL) y tempdb explican la mayoría de incidencias que veremos en los módulos siguientes.

02

MÓDULO 2 · 5,5 h

Gestión y Seguridad

DDL, DML y modelo de seguridad de SQL Server

- Tablas, restricciones y vistas
- Procedimientos almacenados y plan cache
- JOINs, subconsultas y window functions
- Logins, users, roles y permisos
- Ownership chaining y mínimo privilegio
- Laboratorio 2: consultas y permisos

DDL

Creación de tablas y restricciones

T-SQL · BD Ventas

```
CREATE TABLE dbo.Cliente (
  ClienteId INT IDENTITY(1,1) NOT NULL,
  Email     NVARCHAR(150) NOT NULL,
  Nombre    NVARCHAR(100) NOT NULL,
  Alta      DATETIME2(0)  NOT NULL
            CONSTRAINT DF_Cliente_Alta DEFAULT SYSDATETIME(),
  CONSTRAINT PK_Cliente PRIMARY KEY (ClienteId),
  CONSTRAINT UQ_Cliente_Email UNIQUE (Email)
);

CREATE TABLE dbo.Pedido (
  PedidoId  INT IDENTITY(1,1) NOT NULL,
  ClienteId INT NOT NULL,
  Fecha     DATE NOT NULL,
  Total     DECIMAL(12,2) NOT NULL,
  CONSTRAINT PK_Pedido PRIMARY KEY (PedidoId),
  CONSTRAINT FK_Pedido_Cliente FOREIGN KEY (ClienteId)
            REFERENCES dbo.Cliente (ClienteId),
  CONSTRAINT CK_Pedido_Total CHECK (Total >= 0)
);
```



Las cinco restricciones

- **PK:** identifica la fila; crea índice clustered
- **FK:** integridad referencial; sin índice automático
- **UNIQUE:** valores no repetidos (índice único)
- **CHECK:** regla de dominio por fila
- **DEFAULT:** valor si se omite la columna

BUENAS PRÁCTICAS

Errores comunes en el diseño DDL

Tabla sin clave primaria

Un heap sin PK admite duplicados y complica replicación y CDC. Define siempre una PK.

FK sin índice de apoyo

SQL Server no indexa la FK: un DELETE en el padre escanea la tabla hija y bloquea.

Restricciones WITH NOCHECK

La restricción queda «no confiable» y el optimizador deja de aprovecharla.

VARCHAR mezclado con NVARCHAR

La conversión implícita puede impedir un Index Seek y dispara la CPU.

GUID aleatorio como clustered

NEWID() fragmenta el índice con Page Splits. Prefiere IDENTITY o NEWSEQUENTIALID().

Tipos sobredimensionados

Más páginas, más I/O y más memoria en el Buffer Pool para el mismo dato.

Vistas estándar frente a vistas indexadas

Vista estándar

- Consulta guardada, sin datos propios
- Se expande dentro de la consulta externa
- Simplifica joins y centraliza permisos
- Coste = el de la consulta subyacente

Vista indexada

- Resultado materializado en un índice
- Se mantiene en cada INSERT, UPDATE y DELETE
- Acelera agregaciones repetitivas
- Exige SCHEMABINDING y penaliza escrituras

T-SQL · vista estándar

```
CREATE VIEW dbo.vPedidoCliente AS
SELECT c.ClienteId, c.Nombre, p.PedidoId, p.Fecha, p.Total
FROM   dbo.Cliente AS c JOIN dbo.Pedido AS p ON p.ClienteId = c.ClienteId;
```

Vistas indexadas: SCHEMABINDING y requisitos

T-SQL · vista indexada

```
CREATE VIEW dbo.vVentasCliente
WITH SCHEMABINDING
AS
SELECT  p.ClienteId,
        COUNT_BIG(*)  AS NumPedidos,
        SUM(p.Total)  AS TotalVentas
FROM    dbo.Pedido AS p
GROUP BY p.ClienteId;
GO
CREATE UNIQUE CLUSTERED INDEX IX_vVentasCliente
    ON dbo.vVentasCliente (ClienteId);
GO
SELECT * FROM dbo.vVentasCliente WITH (NOEXPAND);
```

Coste: cada DML sobre dbo.Pedido también actualiza la vista.

Requisitos principales

- **SCHEMABINDING** y nombres de dos partes
- **COUNT_BIG(*)** si hay GROUP BY
- Sin OUTER JOIN, subconsultas, DISTINCT, TOP, UNION ni MIN/MAX
- **Primer índice:** UNIQUE CLUSTERED
- Opciones SET: ARITHABORT, ANSI_NULLS y QUOTED_IDENTIFIER en ON
- **Standard:** usar NOEXPAND

PROGRAMABILIDAD

Procedimientos almacenados

T-SQL · procedimiento

```
CREATE OR ALTER PROCEDURE dbo.usp_PedidosPorCliente
    @ClienteId INT,
    @Desde     DATE = '19000101'
AS
BEGIN
    SET NOCOUNT ON;
    SELECT p.PedidoId, p.Fecha, p.Total
    FROM   dbo.Pedido AS p
    WHERE  p.ClienteId = @ClienteId
      AND  p.Fecha >= @Desde
    ORDER BY p.Fecha DESC;
END;
GO
EXEC dbo.usp_PedidosPorCliente @ClienteId = 42;
```



Por qué usarlos

- Plan reutilizable en el Plan Cache
- EXECUTE sin dar acceso a las tablas
- Lógica encapsulada con TRY...CATCH
- SET NOCOUNT ON reduce tráfico
- Parámetros tipados frente a SQL concatenado

PROGRAMABILIDAD

Plan caching, sniffing y recompilaciones

1

Compilar

El optimizador crea el plan con estadísticas y el valor del parámetro.

2

Cachear

El plan se guarda en el Plan Cache, localizado por hash del texto.

3

Reutilizar

Las siguientes ejecuciones se saltan la compilación.

4

Recompilar

Cambios de esquema, estadísticas o SET options lo invalidan.

T-SQL · mitigaciones y DMV

```
-- Mitigaciones de parameter sniffing
EXEC dbo.usp_PedidosPorCliente @ClienteId = 42 WITH RECOMPILE;
-- En la consulta: OPTION (RECOMPILE) / OPTION (OPTIMIZE FOR UNKNOWN)
SELECT TOP (5) usecounts, size_in_bytes, objtype
FROM sys.dm_exec_cached_plans ORDER BY usecounts DESC;
```

Sniffing: el primer valor compilado fija el plan para todos los demás.

SELECT complejos: orden lógico de ejecución

1. **FROM** — Origen y JOINs
2. **WHERE** — Filtra filas
3. **GROUP BY** — Agrupa filas
4. **HAVING** — Filtra grupos
5. **SELECT** — Columnas y OVER
6. **ORDER BY** — Ordena y TOP

T-SQL · ventas por cliente y año

```
SELECT   c.Nombre, YEAR(p.Fecha) AS Anio, SUM(p.Total) AS Ventas
FROM     dbo.Cliente AS c
JOIN     dbo.Pedido  AS p ON p.ClienteId = c.ClienteId
WHERE    p.Fecha >= '20240101'
GROUP BY c.Nombre, YEAR(p.Fecha)
HAVING   SUM(p.Total) > 1000
ORDER BY Ventas DESC;
```

Alias: ORDER BY puede usar alias de SELECT porque se evalúa después; WHERE no.

JOINs: tipos lógicos y trampas habituales

Tipo	Devuelve	Uso típico
INNER	Sólo coincidencias	Pedidos con su cliente
LEFT	Toda la izquierda + NULL	Clientes sin pedidos
RIGHT	Toda la derecha + NULL	Poco usado (invertir)
FULL	Ambos lados completos	Conciliaciones
CROSS	Producto cartesiano	Calendarios, combinaciones

Trampa: filtrar la tabla derecha en el WHERE convierte el LEFT JOIN en INNER.

T-SQL · LEFT JOIN

```
-- 1) Anti-join: clientes sin pedidos
SELECT c.ClienteId, c.Nombre
FROM   dbo.Cliente AS c
LEFT JOIN dbo.Pedido AS p
       ON p.ClienteId = c.ClienteId
WHERE  p.PedidoId IS NULL;

-- 2) Filtra la tabla derecha en el ON
SELECT c.Nombre, p.PedidoId
FROM   dbo.Cliente AS c
LEFT JOIN dbo.Pedido AS p
       ON p.ClienteId = c.ClienteId
      AND p.Fecha >= '20240101';
```

JOINs: algoritmos físicos del optimizador

Nested Loops

- Por cada fila externa busca en la interna
- Ideal: externa pequeña + Index Seek
- Típico en OLTP

Merge Join

- Recorre dos entradas ordenadas
- Exige igualdad y orden (índice o Sort)
- Eficiente con grandes volúmenes

Hash Match

- Tabla hash con la entrada pequeña
- No exige índices ni orden
- Usa memory grant; puede derramar a tempdb

En el plan: compara Estimated frente a Actual Rows. Estadísticas obsoletas provocan joins inadecuados.

Subconsultas, EXISTS y CTE

T-SQL · subconsulta y EXISTS

```
-- Subconsulta escalar
SELECT PedidoId, Total
FROM   dbo.Pedido
WHERE  Total > (SELECT AVG(Total)
                FROM dbo.Pedido);

-- EXISTS (correlacionada)
SELECT c.Nombre
FROM   dbo.Cliente AS c
WHERE  EXISTS (SELECT 1 FROM dbo.Pedido AS p
               WHERE p.ClienteId = c.ClienteId);
```

T-SQL · CTE

```
WITH VentasCliente AS (
    SELECT ClienteId, SUM(Total) AS Ventas
    FROM   dbo.Pedido
    GROUP BY ClienteId
)
SELECT TOP (5) c.Nombre, v.Ventas
FROM   VentasCliente AS v
JOIN   dbo.Cliente  AS c ON c.ClienteId = v.ClienteId
ORDER BY v.Ventas DESC;
```



Recuerda: NOT IN con algún NULL devuelve cero filas: usa NOT EXISTS. Una CTE no materializa datos: se expande como una vista en línea.

Agregados y window functions básicas

T-SQL · window functions

```
SELECT ClienteId, Fecha, Total,
  ROW_NUMBER() OVER (PARTITION BY ClienteId
                     ORDER BY Fecha DESC) AS rn,
  RANK()       OVER (ORDER BY Total DESC) AS ranking,
  SUM(Total)   OVER (PARTITION BY ClienteId
                     ORDER BY Fecha
                     ROWS UNBOUNDED PRECEDING) AS acumulado,
  LAG(Total)   OVER (PARTITION BY ClienteId
                     ORDER BY Fecha) AS anterior
FROM dbo.Pedido;
```

Función	Resultado
ROW_NUMBER	1, 2, 3 sin empates
RANK	Empates comparten puesto; hay huecos
SUM OVER	Total acumulado por cliente
LAG	Valor de la fila anterior

GROUP BY vs OVER: GROUP BY colapsa filas; OVER mantiene el detalle. Indica ROWS en el frame para evitar el worktable de RANGE.

Laboratorio 2A: consultas y procedimientos

1

Preparar datos

Crea Ventas con Cliente y Pedido y carga 50 000 pedidos sesgados.

2

Consulta analítica

Top 3 pedidos por cliente con ROW_NUMBER y SUM OVER.

3

Procedimiento

Crea usp_TopPedidosCliente con parámetros y SET NOCOUNT ON.

4

Plan y sniffing

Ejecuta con cliente 1 y con otro; compara planes con Ctrl+M.

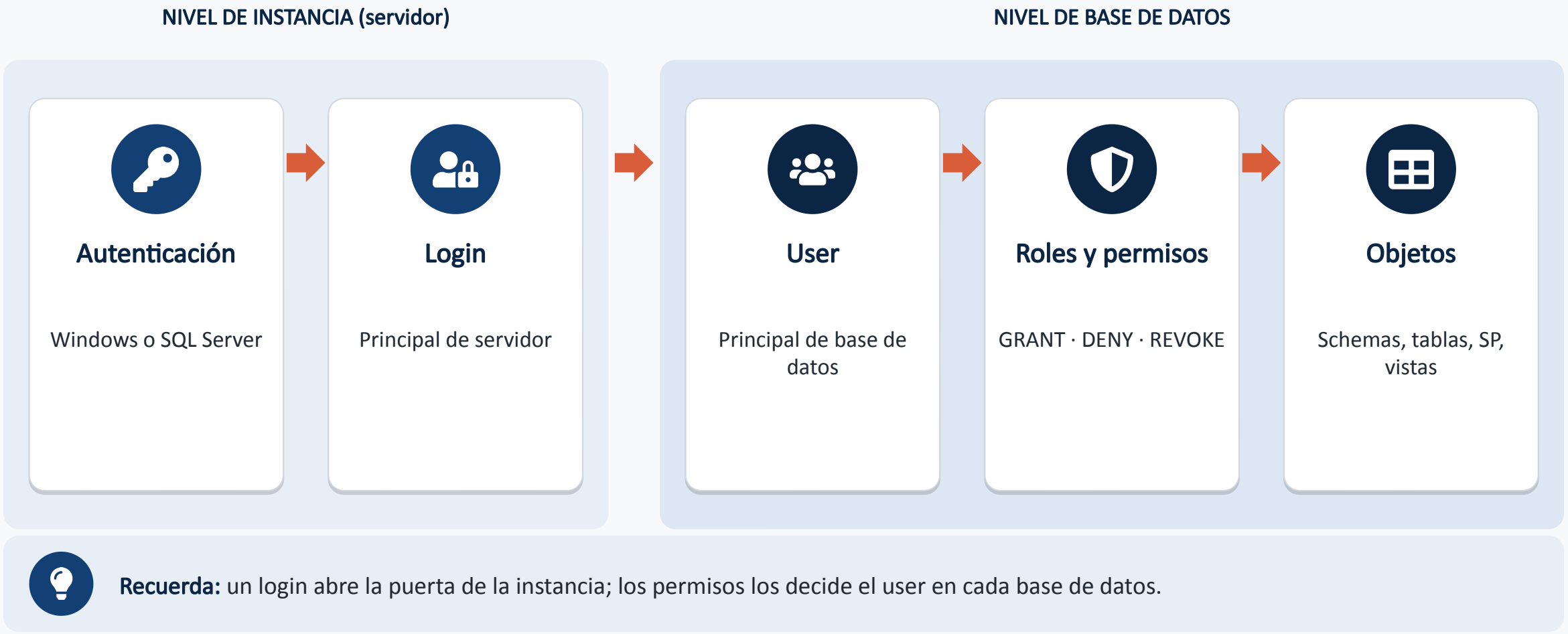
T-SQL · punto de partida del paso 2

```
WITH R AS (SELECT ClienteId, PedidoId, Total,
   ROW_NUMBER() OVER (PARTITION BY ClienteId
                      ORDER BY Total DESC) AS rn
   FROM dbo.Pedido)
SELECT * FROM R WHERE rn <= 3;
```

Trampa: no puedes filtrar rn en el mismo SELECT; envuélvelo en una CTE.

SEGURIDAD

Modelo de seguridad de SQL Server



Recuerda: un login abre la puerta de la instancia; los permisos los decide el user en cada base de datos.

SEGURIDAD

Autenticación: Windows frente a SQL Server

Aspecto	Autenticación de Windows	Autenticación de SQL Server
Identidad	Cuenta o grupo de Active Directory	Login y contraseña guardados en master
Seguridad	Kerberos/NTLM, caducidad y MFA en AD	CHECK_POLICY; riesgo de fuerza bruta (sa)
Recomendado	Opción preferida	Apps heredadas, Linux/Docker, fuera de dominio

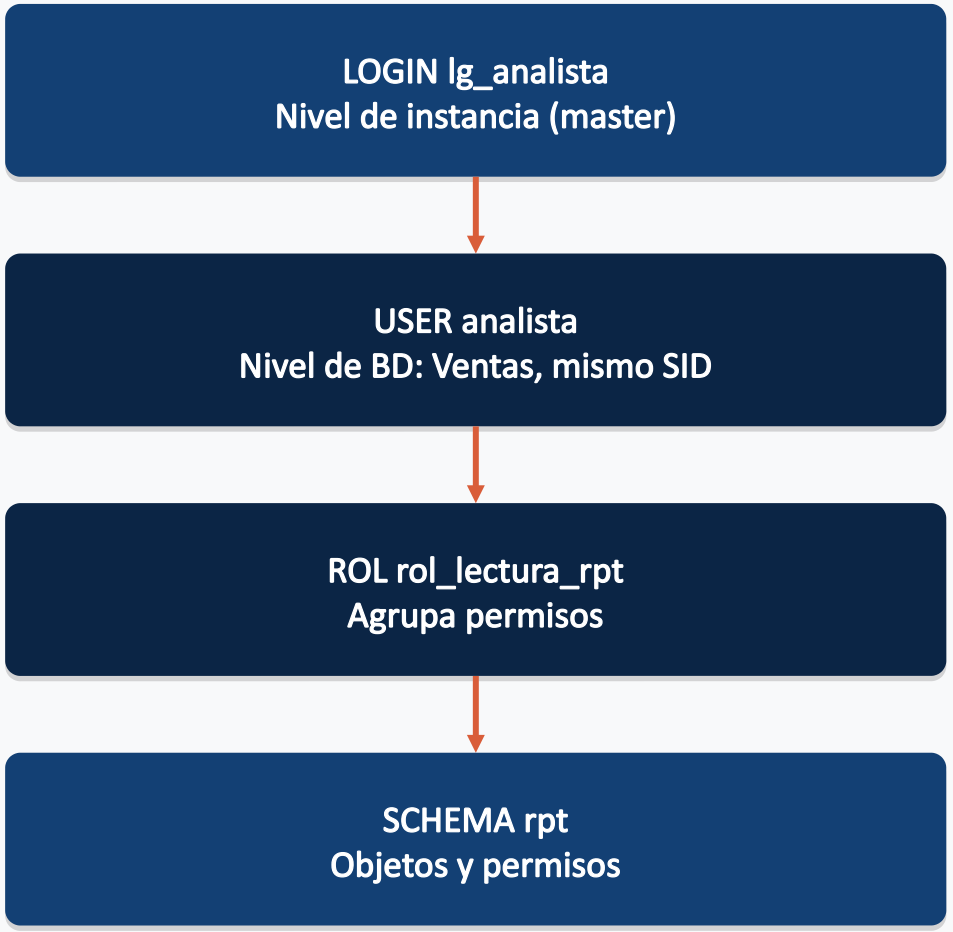
T-SQL · modo y política

```
-- 1 = sólo Windows, 0 = modo mixto
SELECT SERVERPROPERTY('IsIntegratedSecurityOnly') AS SoloWindows;
SELECT name, is_policy_checked, is_expiration_checked
FROM   sys.sql_logins;
```

Modo mixto: requiere reiniciar el servicio y deja sa expuesta: deshabilítala.

SEGURIDAD

Logins, Users y Schemas



T-SQL · login, schema y user

```
-- 1) Instancia: el login
CREATE LOGIN lg_analista
    WITH PASSWORD = N'C0mpl3ja!2024', CHECK_POLICY = ON;

-- 2) Base de datos: schema y user mapeado
USE Ventas;
GO
CREATE SCHEMA rpt AUTHORIZATION dbo;
GO
CREATE USER analista FOR LOGIN lg_analista
    WITH DEFAULT_SCHEMA = rpt;
```

Huérfano: tras un restore, ALTER USER ... WITH LOGIN = ... repara el SID.

SEGURIDAD

Roles fijos y roles personalizados

Rol	Ámbito	Capacidad
sysadmin	Servidor	Control total de la instancia
securityadmin	Servidor	Gestiona logins y permisos
dbcreator	Servidor	Crea y restaura bases de datos
db_owner	Base de datos	Control total sobre la BD
db_datareader / db_datawriter	Base de datos	SELECT / DML en todas las tablas
db_ddladmin	Base de datos	Ejecuta DDL (CREATE, ALTER)

T-SQL · rol personalizado

```
CREATE ROLE rol_lectura_rpt;
GRANT SELECT ON SCHEMA::rpt
    TO rol_lectura_rpt;

CREATE ROLE rol_operador;
GRANT EXECUTE ON SCHEMA::app
    TO rol_operador;

ALTER ROLE rol_lectura_rpt
    ADD MEMBER analista;
```

Mejor práctica: evita db_owner y sysadmin en aplicaciones; usa roles personalizados con permisos sobre un schema.

SEGURIDAD

GRANT, DENY y REVOKE

GRANT

Concede un permiso; WITH GRANT OPTION permite delegarlo.

DENY

Prohíbe el permiso y gana siempre al GRANT, también el heredado.

REVOKE

Quita un GRANT o un DENY previo; no equivale a denegar.

T-SQL · permisos sobre dbo.Pedido

```
GRANT SELECT, INSERT ON dbo.Pedido TO rol_operador;
DENY  DELETE         ON dbo.Pedido TO rol_operador;
REVOKE INSERT        ON dbo.Pedido FROM rol_operador;

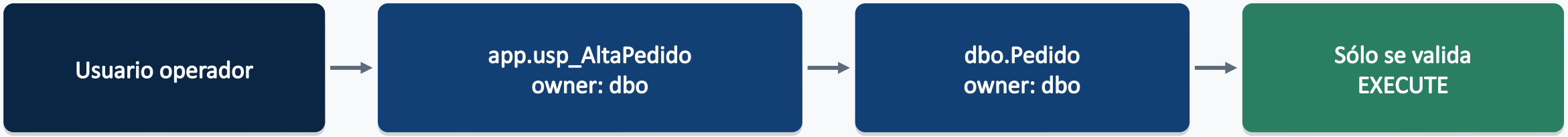
-- Permisos efectivos del contexto actual
SELECT * FROM sys.fn_my_permissions(N'dbo.Pedido', N'OBJECT');
```

Precedencia: DENY > GRANT > sin permiso. Los roles se acumulan, salvo DENY.

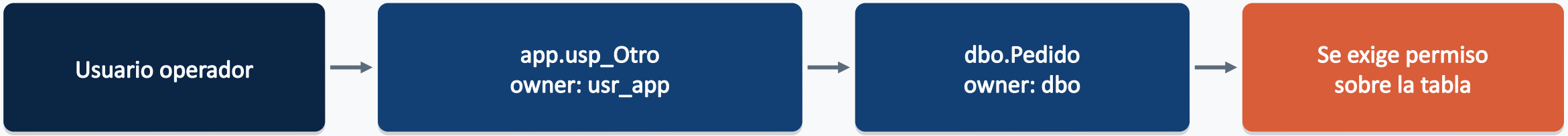
SEGURIDAD

Ownership chaining (cadena de propiedad)

Cadena intacta (mismo propietario)



Cadena rota (propietarios distintos)



Utilidad: da EXECUTE sobre el procedimiento, no acceso a las tablas: es una API controlada.

Límites: el SQL dinámico rompe la cadena; entre bases de datos exige DB_CHAINING.

BUENAS PRÁCTICAS

Mínimo privilegio y auditoría básica

Mínimo privilegio

- Permisos a roles, no a usuarios sueltos
- GRANT sobre schemas o procedimientos
- Cuentas de aplicación ≠ administración
- sa deshabilitada y sysadmin revisado

Auditoría básica

- SQL Server Audit a archivo o Event Log
- Login auditing: fallos y éxitos
- DMVs: sys.dm_exec_sessions, sys.server_principals
- Triggers DDL / Extended Events

T-SQL · SQL Server Audit

```
CREATE SERVER AUDIT Aud_Seguridad
    TO FILE (FILEPATH = N'/var/opt/mssql/audit/');
CREATE SERVER AUDIT SPECIFICATION Aud_Seg_Spec
    FOR SERVER AUDIT Aud_Seguridad
    ADD (FAILED_LOGIN_GROUP) WITH (STATE = ON);
ALTER SERVER AUDIT Aud_Seguridad WITH (STATE = ON);
```

Lectura: consulta los eventos con sys.fn_get_audit_file.

LABORATORIO · PERMISOS

Laboratorio 2B: permisos segregados

1

Crear principals

Dos logins, sus users en Ventas y los schemas rpt y app.

2

Definir roles

rol_lectura_rpt sobre rpt y rol_operador con EXECUTE sobre app.

3

Asignar y denegar

Añade miembros y un DENY SELECT sobre dbo.Pedido al operador.

4

Probar

EXECUTE AS USER, REVERT y sys.fn_my_permissions.

T-SQL · verificación

```
EXECUTE AS USER = 'operador';
EXEC app.usp_AltaPedido @ClienteId = 1, @Total = 99.90; -- OK
SELECT TOP (1) * FROM dbo.Pedido;                       -- error 229
REVERT;
```

Trampa: olvidar REVERT deja la sesión suplantada.

RESUMEN

Resumen del Módulo 2: puntos clave

DDL e integridad

- PK, FK, CHECK, UNIQUE, DEFAULT
- FK con índice de apoyo
- Tipos ajustados al dato

Vistas y SP

- Vistas indexadas con SCHEMABINDING
- Plan cache y sniffing
- SP como API segura

DML analítico

- Orden lógico del SELECT
- Nested, Merge y Hash
- OVER, RANK y LAG

Seguridad

- Login, user, rol, permiso
- DENY gana a GRANT
- Mínimo privilegio y auditoría

Siguiente módulo: índices y B-Tree, planes de ejecución, DMVs de rendimiento y alta disponibilidad (RTO/RPO).

03

MÓDULO 3 · 5 h

Optimización y Alta Disponibilidad

Índices, planes de ejecución, DMVs y fundamentos de HA/DR

- Arquitectura de índices y B-Tree
- Fragmentación y estadísticas
- Lectura de planes de ejecución
- Monitorización con DMVs
- RTO, RPO y comparativa HA/DR
- Laboratorio 3

Heap vs Clustered Index

Heap (sin índice clustered)

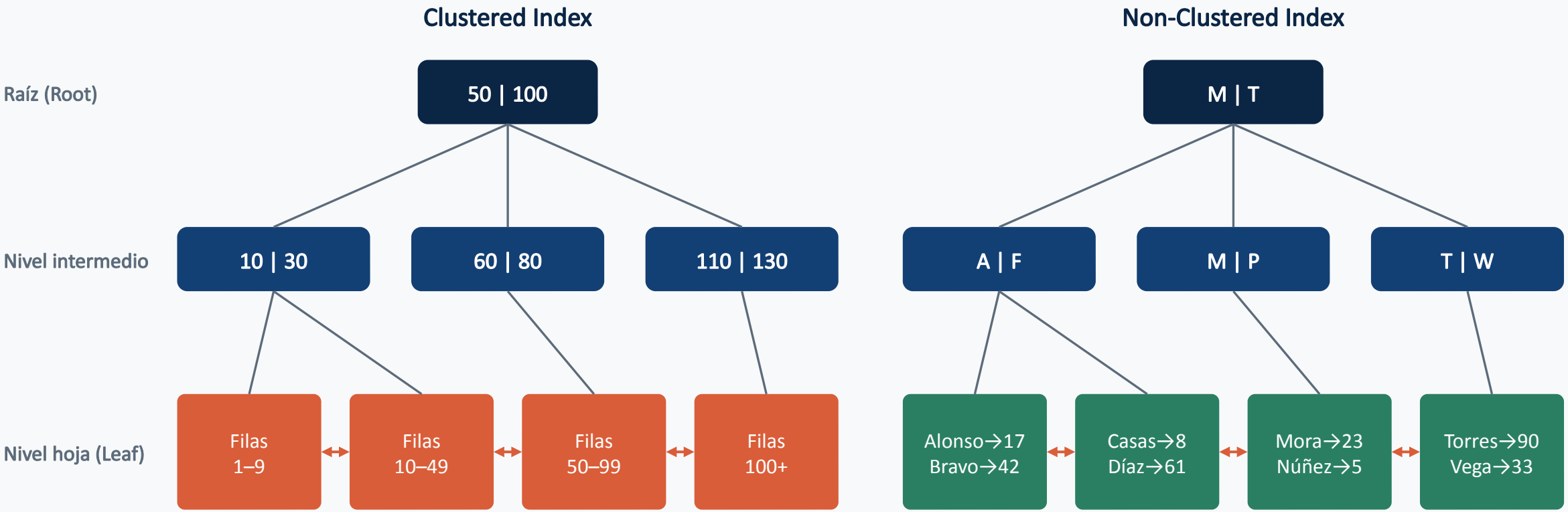
- Filas en páginas sin orden lógico (se localizan por IAM)
- Lectura completa = Table Scan; búsqueda por RID
- Forwarded Records tras UPDATE que hace crecer la fila
- Útil en staging y cargas masivas puntuales

Clustered Index

- La hoja del B-Tree contiene las filas, ordenadas por la clave
- Sólo uno por tabla (la PRIMARY KEY lo crea por defecto)
- Clave ideal: estrecha, única, estática y creciente
- Sin Forwarded Records, pero con posibles Page Splits

Regla práctica: en OLTP casi toda tabla debe tener un Clustered Index estrecho y creciente (por ejemplo INT IDENTITY). Cada columna de esa clave se copia en todos los índices no clustered.

Estructura B-Tree de un índice



La hoja = páginas de datos (la tabla ES el índice). Hojas enlazadas en lista doble.

La hoja guarda clave + puntero (clave clustered o RID) → Key Lookup.

Non-Clustered, Key Lookup y covering

T-SQL · CREAR ÍNDICES

```sql
-- Consulta frecuente
SELECT PedidoId, FechaPedido, Total
FROM dbo.Pedido
WHERE ClienteId = 4217;

-- 1) Sólo clave: Seek + Key Lookup por fila
CREATE NONCLUSTERED INDEX IX_Pedido_Cliente
    ON dbo.Pedido (ClienteId);

-- 2) Covering: sin Key Lookup
CREATE NONCLUSTERED INDEX IX_Pedido_Cliente_Cov
    ON dbo.Pedido (ClienteId)
    INCLUDE (FechaPedido, Total);
```

Qué ocurre por dentro

- La hoja del NCI guarda clave + puntero a la fila
- Key Lookup: 1 lectura aleatoria por fila
- INCLUDE: columnas sólo en la hoja, sin ordenar
- Más índices = más coste en INSERT/UPDATE/DELETE

Orden de la clave: igualdad primero, rango después, y columnas de lectura en INCLUDE.

Filtered, columnstore y missing indexes

T-SQL · FILTERED Y COLUMNSTORE

```
-- Filtered: sólo pedidos abiertos
CREATE NONCLUSTERED INDEX IX_Pedido_Abierto
    ON dbo.Pedido (FechaPedido)
    INCLUDE (Total)
    WHERE Estado = 1;
-- Columnstore (analítica / DW)
CREATE NONCLUSTERED COLUMNSTORE INDEX
    NCCI_Pedido ON dbo.Pedido (ClienteId, Total);
```

T-SQL · MISSING INDEX DMVS

```
SELECT TOP (5)
  d.statement AS tabla,
  d.equality_columns, d.inequality_columns,
  d.included_columns,
  s.user_seeks, s.avg_user_impact
FROM sys.dm_db_missing_index_details AS d
JOIN sys.dm_db_missing_index_groups AS g
  ON g.index_handle = d.index_handle
JOIN sys.dm_db_missing_index_group_stats AS s
  ON s.group_handle = g.index_group_handle
ORDER BY s.user_seeks * s.avg_user_impact DESC;
```

Cuándo usar cada uno

- Filtered: subconjuntos pequeños, predicado fijo
- Columnstore: agregaciones sobre millones de filas
- Filtered + parámetros: puede ignorarse sin RECOMPILE

Con cautela: son sugerencias, no órdenes. Pueden duplicar índices y se reinician con el servicio. Valídalas con el plan real.

MANTENIMIENTO

Fragmentación, Page Splits y fillfactor

T-SQL · MEDIR FRAGMENTACIÓN

```
SELECT OBJECT_NAME(ips.object_id) AS Tabla,
       i.name AS Indice,
       ips.avg_fragmentation_in_percent AS Frag,
       ips.page_count
FROM sys.dm_db_index_physical_stats
     (DB_ID(), NULL, NULL, NULL, N'LIMITED') AS ips
JOIN sys.indexes AS i
  ON i.object_id = ips.object_id
 AND i.index_id = ips.index_id
WHERE ips.page_count > 1000
  AND ips.avg_fragmentation_in_percent > 5
ORDER BY Frag DESC;
```

T-SQL · REORGANIZE / REBUILD

```
ALTER INDEX IX_Pedido_Cliente ON dbo.Pedido REORGANIZE;
ALTER INDEX IX_Pedido_Cliente ON dbo.Pedido
    REBUILD WITH (FILLFACTOR = 90);
```

Fragmentación	Acción
< 5 %	No hacer nada
5 – 30 %	REORGANIZE
> 30 %	REBUILD

Page Split

- Página llena + inserción en medio: se parte en dos
- Deja páginas a medias y genera log extra
- Claves aleatorias (GUID) lo agravan

OPTIMIZADOR

Estadísticas y cardinality estimator

T-SQL · INSPECCIONAR ESTADÍSTICAS

```
-- Opciones a nivel de base de datos
SELECT name, is_auto_create_stats_on,
       is_auto_update_stats_on,
       is_auto_update_stats_async_on
FROM sys.databases WHERE name = N'Ventas';

-- Encabezado, densidad e histograma
DBCC SHOW_STATISTICS (N'dbo.Pedido',
                      N'IX_Pedido_Cliente');

-- Actualización manual
UPDATE STATISTICS dbo.Pedido IX_Pedido_Cliente
    WITH FULLSCAN;
```

Auto-create / Auto-update

Se refrescan tras muchos cambios: umbral dinámico √(1000·filas) con compat. 130+.

Histograma

Hasta 200 pasos sobre la 1.ª columna de la clave: filas estimadas por valor.

Cardinality Estimator

El CE nuevo (compat. 120+) cambia estimaciones; compara con el Legacy CE.

Planes: Scan, Seek y Key Lookup

Scan

- Lee todas las páginas del objeto
- Coste proporcional al tamaño (E/S)
- Correcto en tablas pequeñas o lecturas masivas
- Sospechoso: predicado no SARGable

Seek

- Navega el B-Tree hasta la clave
- Pocas lecturas lógicas
- Ideal con predicados selectivos
- Ojo: devolver millones de filas no es barato

Key Lookup

- Un acceso al clustered por fila del NCI
- Lecturas aleatorias que se multiplican
- Plan típico: Nested Loops + Lookup
- Solución: INCLUDE o reescribir la consulta

Un Scan no es siempre un problema: hay que cruzarlo con el nº de filas, el tamaño de la tabla y las lecturas lógicas reales antes de decidir crear un índice.

Leer un plan: coste, warnings y estimados

T-SQL · MEDIR E/S Y TIEMPO

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SELECT PedidoId, Total
FROM dbo.Pedido
WHERE ClienteId = 4217;

-- Messages: Table 'Pedido'. Scan count 1,
--   logical reads 3, physical reads 0
--   CPU time = 0 ms, elapsed time = 1 ms
```

Métrica clave: las lecturas lógicas son estables; el tiempo varía con la caché y la carga.

Qué mirar en el plan

- **Estimated vs Actual:** gran diferencia = estadísticas obsoletas o parameter sniffing
- **Warnings:** spill a tempdb, conversión implícita, falta de estadísticas
- **Coste %:** estimación relativa, no tiempo real
- **Query Store:** planes e historial para detectar regresiones (2016+)

MONITORIZACIÓN

DMVs: consultas costosas y sesiones activas

T-SQL · TOP CONSULTAS POR CPU

```
SELECT TOP (5)
  qs.execution_count,
  qs.total_worker_time/qs.execution_count AS cpu_us,
  qs.total_logical_reads,
  LEFT(st.text, 100) AS batch_text
FROM sys.dm_exec_query_stats AS qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
-- Plan: sys.dm_exec_query_plan(qs.plan_handle)
ORDER BY qs.total_worker_time DESC;
```

T-SQL · PETICIONES ACTIVAS

```
SELECT r.session_id, r.status, r.command,
       r.wait_type, r.wait_time,
       r.blocking_session_id,
       r.cpu_time, r.logical_reads,
       DB_NAME(r.database_id) AS bd,
       LEFT(t.text, 100) AS sql_text
FROM sys.dm_exec_requests AS r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.session_id <> @@SPID;
```



Datos volátiles: las DMVs acumulan desde el último reinicio y query_stats sólo ve planes en caché. Necesitas VIEW SERVER STATE (VIEW SERVER PERFORMANCE STATE en 2022).

DMVs: esperas y bloqueos

T-SQL · TOP WAITS ACUMULADOS

```
SELECT TOP (10)
  wait_type,
  waiting_tasks_count,
  wait_time_ms,
  signal_wait_time_ms,
  CAST(100.0 * wait_time_ms /
    SUM(wait_time_ms) OVER() AS DECIMAL(5,2)) AS pct
FROM sys.dm_os_wait_stats
WHERE wait_type NOT LIKE N'SLEEP%'
  AND wait_type NOT IN (N'LAZYWRITER_SLEEP',
                        N'XE_TIMER_EVENT')
ORDER BY wait_time_ms DESC;
```

T-SQL · TAREAS EN ESPERA AHORA

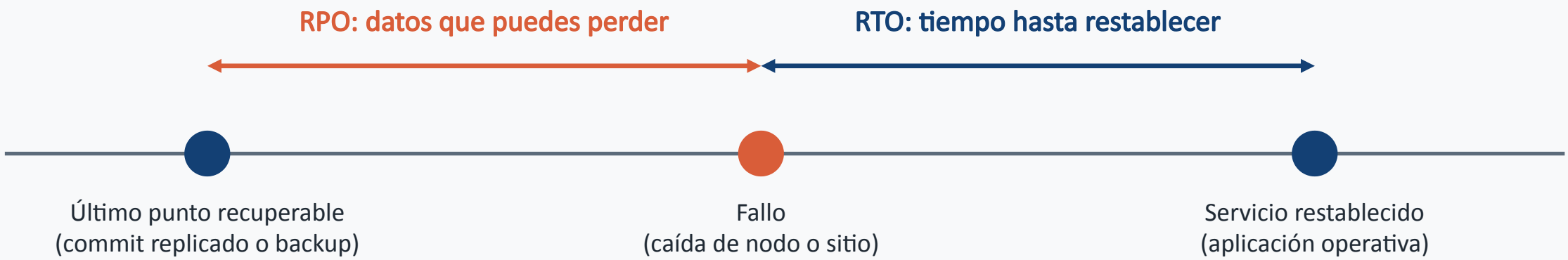
```
SELECT wt.session_id, wt.wait_type,
       wt.wait_duration_ms,
       wt.blocking_session_id,
       wt.resource_description
FROM sys.dm_os_waiting_tasks AS wt
WHERE wt.session_id > 50
ORDER BY wt.wait_duration_ms DESC;
```

Consejo: toma dos snapshots y calcula la diferencia.

Wait type	Suele indicar
PAGEIOLATCH_*	Lecturas de disco lentas
LCK_M_*	Bloqueos entre sesiones
SOS_SCHEDULER_YIELD	Presión de CPU
WRITELOG	Latencia del disco de log

HA / DR

RTO y RPO: tiempo y datos en juego



RPO ≈ 0

FCI o AG síncrono: ningún commit confirmado se pierde

RTO: segundos

Failover automático con FCI o AG síncrono y listener

RPO: minutos

Log Shipping: pérdida = intervalo de backup del log

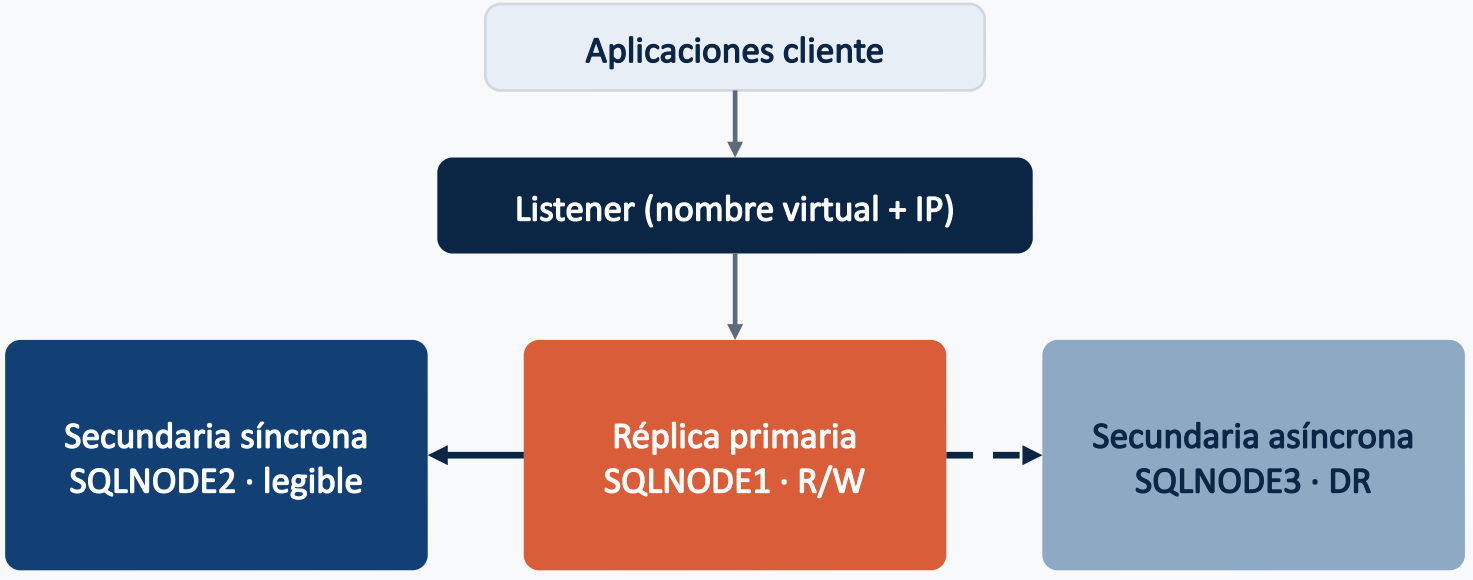
Comparativa de tecnologías HA/DR

Tecnología	Alcance	Failover	Pérdida de datos (RPO)	Punto clave
Log Shipping	Base de datos	Manual	Minutos (intervalo de log backup)	Simple y barato; secundaria STANDBY
Replicación	Tablas / artículos	Manual (no es HA)	Variable (latencia de los agentes)	Distribuye datos; no es DR completo
FCI	Instancia completa	Automático (WSFC)	≈ 0 (almacenamiento compartido)	Requiere SAN; el disco es punto único
Always On AG	Grupo de bases de datos	Automático (síncrona) o manual	≈ 0 síncrona · > 0 asíncrona	Sin disco compartido; réplicas legibles



Elige por RTO/RPO, edición y presupuesto: Standard ofrece Basic AG, FCI de 2 nodos y Log Shipping; Enterprise añade AG completos.

Always On AG: topología y quorum



Windows Server Failover Cluster (WSFC)
Quorum: voto por nodo + Witness (File Share / Cloud / Disk)

Basic AG (Standard)

2 réplicas, 1 BD por AG, sin lectura en la secundaria.

Distributed AG

Une dos AG en clústeres distintos: DR entre sitios y migraciones.

Quorum y Testigo

Mayoría de votos contra split-brain; testigo en nodos pares.

Lab 3.1 · Índices y medición de lecturas

1

Preparar datos

Genera 500 000 pedidos en la base de datos Ventas.

2

Línea base

Activa STATISTICS IO y plan real (Ctrl+M); anota las lecturas.

3

Crear índices

Crea el índice simple y luego la versión con INCLUDE.

4

Comparar

Repite la consulta y compara operador y lecturas lógicas.

T-SQL · LÍNEA BASE

```
USE Ventas;
SET STATISTICS IO, TIME ON;
SELECT PedidoId, FechaPedido, Total
FROM dbo.Pedido
WHERE ClienteId = 4217
  AND FechaPedido >= '20240101';
-- Anota: operador y logical reads
```

T-SQL · CREAR Y MEJORAR EL ÍNDICE

```
CREATE NONCLUSTERED INDEX IX_Pedido_Cli_Fecha
  ON dbo.Pedido (ClienteId, FechaPedido);
-- Lookup: añade INCLUDE y recrea
CREATE NONCLUSTERED INDEX IX_Pedido_Cli_Fecha
  ON dbo.Pedido (ClienteId, FechaPedido)
  INCLUDE (Total)
  WITH (DROP_EXISTING = ON);
```

Lab 3.2 · Analizar planes de ejecución

T-SQL · TRES CONSULTAS, TRES PLANES

```
SET STATISTICS IO ON;
CREATE NONCLUSTERED INDEX IX_Pedido_Fecha
    ON dbo.Pedido (FechaPedido);
-- A) No SARGable: función sobre la columna
SELECT PedidoId FROM dbo.Pedido
WHERE YEAR(FechaPedido) = 2024;
-- B) SARGable: rango
SELECT PedidoId FROM dbo.Pedido
WHERE FechaPedido >= '20240101'
  AND FechaPedido <  '20250101';
-- C) Seek + Key Lookup (pide Total)
SELECT PedidoId, Total FROM dbo.Pedido
WHERE FechaPedido = '20240315';
```

Qué anotar	Dónde verlo
Operador (Scan/Seek)	Plan real (Ctrl+M)
Logical reads	Pestaña Messages
Estimated vs Actual	Propiedades (F4)
Warnings	Triángulo amarillo

Trampa: compara siempre planes reales y repite cada consulta 2–3 veces.

Entrega: tabla con operador, lecturas lógicas y desviación estimado/real de A, B y C.

LABORATORIO · LABORATORIO 3.3

Lab 3.3 · Cuellos de botella con DMVs

1	**Provocar un bloqueo**	Sesión 1 abre una transacción con UPDATE sin COMMIT; Sesión 2 ejecuta un SELECT.
2	**Localizar la espera**	En Sesión 3 consulta sys.dm_exec_requests y sys.dm_os_waiting_tasks (LCK_M_S).
3	**Resolver y revisar**	Identifica al head blocker, haz ROLLBACK y revisa sys.dm_os_wait_stats.

T-SQL · SESIONES 1 Y 2

```
-- Sesión 1: transacción sin cerrar
BEGIN TRAN;
UPDATE dbo.Cliente SET Activo = 0
WHERE ClienteId = 1;
-- Sesión 2: queda bloqueada
SELECT Nombre FROM dbo.Cliente
WHERE ClienteId = 1;
```

T-SQL · SESIÓN 3 (DBA)

```
SELECT r.session_id, r.blocking_session_id,
       r.wait_type, r.wait_time, t.text
FROM sys.dm_exec_requests AS r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.blocking_session_id <> 0;
-- Resolver: ROLLBACK en la sesión 1
```

Módulo 3: puntos clave



Diseño de índices

Clustered estrecho y creciente; NCI con INCLUDE; vigila el coste de escritura.



Diagnóstico

Plan real, STATISTICS IO y DMVs: mide antes de cambiar y compara después.



Mantenimiento

Reorganize 5–30 %, Rebuild > 30 %, estadísticas al día y fillfactor justificado.



HA/DR

Define RTO y RPO primero; elige Log Shipping, FCI o AG según edición y coste.

04

MÓDULO 4 · 5 h

Mantenimiento y Buenas Prácticas

Recuperación, integridad, automatización y resolución de incidencias

- Modelos de recuperación y backups
- Restauración point-in-time
- Integridad física: DBCC CHECKDB
- Mantenimiento automatizado
- Incidencias: LDF, TempDB, bloqueos
- Laboratorio 4 y caso integrador

Modelos de recuperación

Aspecto	Simple	Full	Bulk-Logged
Truncado del log	Automático en cada checkpoint	Sólo tras BACKUP LOG	Sólo tras BACKUP LOG
Log chain	No existe; BACKUP LOG no permitido	Continua; log backups obligatorios	Continua; log backups obligatorios
Operaciones masivas	Minimal logging	Registro completo	Minimal logging (extents)
Restauración posible	Último Full o Diferencial	Cualquier instante (STOPAT)	Cualquier instante, salvo dentro de logs con carga masiva
Uso típico	Desarrollo, DWH recargable	OLTP en producción	Ventanas ETL puntuales



Minimal logging: registra sólo la asignación de extents, no cada fila. Es más rápido y genera menos log, pero un log backup con carga masiva no permite STOPAT en su interior.

Estrategia: Full + Diferencial + Log chain



Secuencia de restauración (STOPAT = 10:36)

1 · RESTORE FULL
Domingo · NORECOVERY

→

2 · RESTORE DIFF
Martes · NORECOVERY

→

3 · RESTORE LOG(s)
Mar 22:15 → Mié 10:30 · NORECOVERY

→

4 · LOG final
WITH STOPAT = 10:36, RECOVERY

T-SQL de backup: Full, Diff y Log

T-SQL

```
-- Full semanal con compresión y verificación de páginas
BACKUP DATABASE Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_FULL.bak'
WITH COMPRESSION, CHECKSUM, INIT, STATS = 10;

-- Diferencial diario (cambios desde el último Full)
BACKUP DATABASE Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_DIFF.bak'
WITH DIFFERENTIAL, COMPRESSION, CHECKSUM;

-- Log cada 15 min (sólo Full / Bulk-Logged)
BACKUP LOG Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_LOG_01.trn'
WITH COMPRESSION, CHECKSUM;

-- Copia puntual que no altera la cadena
BACKUP DATABASE Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_CO.bak'
WITH COPY_ONLY, COMPRESSION, CHECKSUM;
```



COMPRESSION + CHECKSUM

Menos E/S y tamaño; valida páginas al leer.



COPY_ONLY

Copia ad hoc sin alterar diferencial ni log chain.



TO URL (Azure Blob)

Copia externa en la nube con credencial SAS.

Tail-log, verificación e historial msdb

T-SQL

```
-- Tail-log: salvar el final del log antes de restaurar
BACKUP LOG Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_TAIL.trn'
WITH NO_TRUNCATE, NORECOVERY, CHECKSUM;

-- Verificar el backup sin restaurarlo
RESTORE VERIFYONLY
FROM DISK = N'/var/opt/mssql/backup/Ventas_FULL.bak'
WITH CHECKSUM;
```

T-SQL · HISTORIAL EN MSDB

```
SELECT TOP (10) bs.database_name,
       bs.type,   -- D=Full I=Diff L=Log
       bs.backup_finish_date,
       bs.is_copy_only,
       bmf.physical_device_name
FROM msdb.dbo.backupset AS bs
JOIN msdb.dbo.backupmediafamily AS bmf
  ON bmf.media_set_id = bs.media_set_id
WHERE bs.database_name = N'Ventas'
ORDER BY bs.backup_finish_date DESC;
```



VERIFYONLY no basta: sólo comprueba que el fichero es legible. La prueba real es restaurar en otro servidor y ejecutar DBCC CHECKDB; planifícala (p. ej. mensualmente).

RESTAURACIÓN

Restauración point-in-time (STOPAT)

T-SQL

```
-- 1) Full. MOVE reubica los ficheros lógicos
RESTORE DATABASE Ventas
FROM DISK = N'/var/opt/mssql/backup/Ventas_FULL.bak'
WITH NORECOVERY,
  MOVE N'Ventas' TO N'/var/opt/mssql/data/Ventas.mdf',
  MOVE N'Ventas_log' TO N'/var/opt/mssql/data/Ventas.ldf';

-- 2) Último diferencial
RESTORE DATABASE Ventas
FROM DISK = N'/var/opt/mssql/backup/Ventas_DIFF.bak'
WITH NORECOVERY;

-- 3) Logs en orden; el último, con STOPAT
RESTORE LOG Ventas
FROM DISK = N'/var/opt/mssql/backup/Ventas_LOG_01.trn'
WITH NORECOVERY;

RESTORE LOG Ventas
FROM DISK = N'/var/opt/mssql/backup/Ventas_LOG_02.trn'
WITH STOPAT = N'2025-03-12T10:36:00', RECOVERY;
```



NORECOVERY

Deja la BD en RESTORING; permite encadenar más backups.



RECOVERY

Deshace lo no confirmado y abre la BD (ONLINE).



STANDBY

Sólo lectura entre restauraciones; usa fichero undo.

INTEGRIDAD

Integridad física: DBCC CHECKDB

823
Error de E/S del sistema operativo

824
Error lógico: checksum o página rota

825
Lectura con éxito tras reintentos: disco en riesgo

T-SQL

```
-- Verificación de páginas a nivel de BD
ALTER DATABASE Ventas SET PAGE_VERIFY CHECKSUM;

DBCC CHECKDB (N'Ventas') WITH NO_INFOMSGS, ALL_ERRORMSGS;

-- Páginas dañadas registradas por el motor
SELECT database_id, file_id, page_id, event_type
FROM msdb.dbo.suspect_pages;
```



Opciones de reparación

- **REPAIR_REBUILD:** sin pérdida de datos.
- **ALLOW_DATA_LOSS:** último recurso, tras backup.
- Antes: restaurar desde backup.

MANTENIMIENTO

Reorganize vs Rebuild y estadísticas

Criterio	REORGANIZE	REBUILD
Fragmentación	5 – 30 %	> 30 %
Modo	Siempre online	Offline (online en Enterprise)
Interrumpible	Sí, sin perder avance	No (salvo RESUMABLE)
Estadísticas	No las actualiza	Actualiza con FULLSCAN
Log	Moderado y sostenido	Mucho en recovery Full

T-SQL

```
SELECT OBJECT_NAME(ips.object_id) AS Tabla, i.name,
       ips.avg_fragmentation_in_percent AS Frag
FROM sys.dm_db_index_physical_stats
     (DB_ID(), NULL, NULL, NULL, 'LIMITED') AS ips
JOIN sys.indexes AS i
  ON i.object_id = ips.object_id
 AND i.index_id = ips.index_id
WHERE ips.page_count > 1000 AND i.index_id > 0;

ALTER INDEX IX_Pedido_Fecha ON dbo.Pedido REORGANIZE;
ALTER INDEX IX_Pedido_Fecha ON dbo.Pedido REBUILD;
UPDATE STATISTICS dbo.Pedido WITH FULLSCAN;
```



Estadísticas primero: actualízalas por umbral de cambios (dm_db_stats_properties) y en horas valle; suelen aportar más que reconstruir índices.

MANTENIMIENTO

Mantenimiento automatizado con Agent

SQL Server Agent jobs

Pasos T-SQL o CmdExec, schedules e historial en msdb.

Maintenance Plans

Asistente gráfico (SSIS): simple, pero poco flexible.

Scripts Ola Hallengren

DatabaseBackup, IntegrityCheck e IndexOptimize por umbrales.

Limpieza de msdb

sp_delete_backuphistory y sp_purge_jobhistory semanales.

Database Mail y operadores

El operador recibe el correo cuando un job falla.

Alertas del Agent

Gravedad 19–25 y errores 823, 824, 825 y 9002.

INCIDENCIAS

Incidencia: log de transacciones lleno

T-SQL

```
SELECT name, recovery_model_desc,
       log_reuse_wait_desc
FROM sys.databases
WHERE name = N'Ventas';

DBCC SQLPERF (LOGSPACE);
DBCC OPENTRAN (N'Ventas');

BACKUP LOG Ventas
TO DISK = N'/var/opt/mssql/backup/Log2.trn';
```

log_reuse_wait_desc	Causa	Acción
LOG_BACKUP	Sin log backups	BACKUP LOG y programarlo
ACTIVE_TRANSACTION	Transacción larga abierta	DBCC OPENTRAN; cerrarla
AVAILABILITY_REPLICA	Réplica AG atrasada	Revisar red y réplica
REPLICATION	Log Reader parado	Reiniciar el agente



Nunca borres el .ldf: la base puede quedar sin recuperar. Resuelve la causa raíz (log_reuse_wait_desc) y reduce el fichero sólo una vez, después.

Incidencia: TempDB saturada

Version store

RCSI y snapshot guardan versiones; una transacción larga las retiene.

Spills y temporales

Sort/hash sin memoria suficiente y #tablas grandes.

PFS / GAM / SGAM

Contención PAGELATCH en páginas de asignación.

Varios ficheros

Ficheros de igual tamaño: 1 por core, hasta 8.

T-SQL

```
-- ¿Quién consume TempDB?
SELECT TOP (5) session_id,
  internal_objects_alloc_page_count AS p_int
FROM sys.dm_db_session_space_usage
ORDER BY p_int DESC;

-- ¿Contención de asignación?
SELECT session_id, wait_type
FROM sys.dm_os_waiting_tasks
WHERE wait_type LIKE N'PAGELATCH%'
  AND resource_description LIKE N'2:%';
```

Ficheros iguales y pre-dimensionados; no hagas shrink rutinario.

INCIDENCIAS

Bloqueos prolongados y deadlocks

T-SQL

```
-- Peticiones bloqueadas y su bloqueador
SELECT r.session_id, r.blocking_session_id,
       r.wait_type, r.wait_time, t.text
FROM sys.dm_exec_requests AS r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) AS t
WHERE r.blocking_session_id <> 0;

-- Locks en espera
SELECT resource_type, request_mode,
       request_status, request_session_id
FROM sys.dm_tran_locks
WHERE request_status = N'WAIT';

-- Grafo de deadlock (system_health)
SELECT CAST(xe.event_data AS XML) AS grafo
FROM sys.fn_xe_file_target_read_file
     (N'system_health*.xel', NULL, NULL, NULL) AS xe
WHERE xe.object_name = N'xml_deadlock_report';
```



Bloqueo prolongado

Transacción abierta sin COMMIT: el resto espera.



Deadlock (error 1205)

Ciclo de esperas; el Lock Monitor elige víctima.



RCSI y KILL con cautela

Los lectores no bloquean; KILL, el último recurso.

SSIS: arquitectura, motor y herramientas

Motor en el Servidor (Setup)

- **Feature en Setup:** marcar «Integration Services» en el instalador oficial de SQL Server.
- **Servicio de Windows:** registra MsDtsSrvr.exe para gestión y ejecución de paquetes.
- **Requisito CLR:** habilitar sp_configure 'clr enabled', 1; RECONFIGURE en la instancia.
- **Catálogo SSISDB:** crear catálogo en SSMS; protegido por Database Master Key y contraseña.

Entorno de Diseño (Visual Studio)

- **Herramienta IDE:** Visual Studio 2019 o 2022 (Community gratuita, Pro o Enterprise).
- **Extensión oficial:** «SQL Server Integration Services Projects» desde VS Marketplace.
- **Proyectos y artefactos:** soluciones con proyectos .dtproj y paquetes de integración .dtsx.
- **Desacoplamiento:** SSDT ya no viene en la ISO de SQL Server; ciclo de vida independiente.

Desacoplamiento clave: SSDT ya no se incluye en la ISO de SQL Server. El desarrollador diseña paquetes en Visual Studio (.ispac); el DBA gestiona el servicio SSIS, provisiona SSISDB y asegura la ejecución en el servidor.

ETL Y DATA FLOW

Pipeline ETL: ingesta de CSV a SQL Server

1

Flat File Source (CSV)

Conexión al fichero .csv: delimitador de columnas (coma o punto y coma), calificador de texto entre comillas, codificación UTF-8 y salto de línea CRLF. Los datos entran al pipeline como cadenas DT_STR o DT_WSTR.

2

Data Conversion (RAM)

Transformación en memoria: convierte las cadenas de texto del CSV a los tipos de datos relacionales de destino (DT_I4 para enteros, DT_NUMERIC para importes o DT_DATE para fechas). Permite redirigir filas erróneas.

3

OLE DB Destination

Inserción masiva en la tabla de destino de SQL Server. Configuración en modo «Table or view - fast load»: ejecuta un BULK INSERT con TABLOCK, evitando registrar cada fila individualmente en el log de transacciones.

Por qué CSV y Fast Load: el formato CSV es el estándar de la industria para cargas batch limpias y sin dependencias de drivers de Office; la opción Fast Load multiplica por diez la velocidad de inserción al trabajar en bloque.

Despliegue en SSISDB y Jobs con Agent

1. Despliegue del Proyecto (.ispac)

Compilación en Visual Studio (Project Deployment Model) generando el archivo .ispac. Despliegue en SSISDB mediante el asistente de SSMS, organizando carpetas por aplicación.

2. Programación en SQL Server Agent

Creación de un Job con paso (Job Step) de tipo «Integration Services Package». Selección del proyecto y paquete en SSISDB, con horarios (Schedules) y reintentos automáticos.

3. Seguridad y Proxies del Agent

Regla de oro: nunca ejecutar paquetes bajo la cuenta de servicio de SQL Server ni como sysadmin. Uso de Credential de Windows y Proxy de Agent con acceso NTFS mínimo al recurso de CSVs.

4. Monitorización y Diagnóstico

Informes nativos en SSMS con clic derecho en SSISDB («All Executions») para analizar duración y filas procesadas. Consulta de vistas catalog.executions y catalog.operation_messages.

Trampa crítica en producción: un paquete que funciona en Visual Studio falla en el Agent si no se usa un Proxy; la cuenta de servicio de SQL Server no suele tener permisos NTFS sobre las carpetas de red compartidas con los CSVs.

Preparar la BD y la cadena de backups

T-SQL

```
USE master;
ALTER DATABASE Ventas SET RECOVERY FULL;

BACKUP DATABASE Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_L4_FULL.bak'
WITH COMPRESSION, CHECKSUM, INIT;

USE Ventas;
INSERT dbo.Cliente (Nombre) VALUES (N'Lab4 A');
BACKUP LOG Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_L4_LOG1.trn'
WITH CHECKSUM, INIT;

INSERT dbo.Cliente (Nombre) VALUES (N'Lab4 B');
BACKUP DATABASE Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_L4_DIFF.bak'
WITH DIFFERENTIAL, CHECKSUM, INIT;
```

1. **Full en recovery Full**
   ALTER DATABASE y BACKUP DATABASE con CHECKSUM.

2. **Log backup 1**
   Tras insertar datos, inicia la log chain.

3. **Diferencial**
   Tras más actividad: WITH DIFFERENTIAL.

4. **Segundo log backup**
   Un cliente más y BACKUP LOG; anota el COUNT.

Simular la pérdida de datos y el tail-log

1 **Anota HoraOK**
SELECT SYSDATETIME() antes del accidente.

2 **Provoca el error**
DELETE sin WHERE sobre dbo.Pedido.

3 **Salva la cola**
Tail-log con NORECOVERY: la BD pasa a RESTORING.

T-SQL

```
USE Ventas;
SELECT COUNT(*) AS Pedidos FROM dbo.Pedido;
SELECT SYSDATETIME() AS HoraOK;    -- anótala
WAITFOR DELAY '00:00:05';

DELETE FROM dbo.Pedido;            -- "accidente"

USE master;
BACKUP LOG Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_L4_TAIL.trn'
WITH NORECOVERY, CHECKSUM, INIT;
```

Trampa: si hay conexiones abiertas a Ventas, usa ALTER DATABASE … SET SINGLE_USER WITH ROLLBACK IMMEDIATE antes del tail-log.

Restaurar a un punto en el tiempo

T-SQL

```
USE master;
RESTORE DATABASE Ventas
FROM DISK = N'/var/opt/mssql/backup/Ventas_L4_FULL.bak'
WITH NORECOVERY, REPLACE;

RESTORE DATABASE Ventas
FROM DISK = N'/var/opt/mssql/backup/Ventas_L4_DIFF.bak'
WITH NORECOVERY;

RESTORE LOG Ventas
FROM DISK = N'/var/opt/mssql/backup/Ventas_L4_LOG1.trn'
WITH NORECOVERY;

RESTORE LOG Ventas
FROM DISK = N'/var/opt/mssql/backup/Ventas_L4_TAIL.trn'
WITH STOPAT = N'2025-03-12T10:36:00', RECOVERY;

SELECT COUNT(*) AS Pedidos FROM Ventas.dbo.Pedido;
```

Verifica el resultado

- COUNT(*) igual al baseline
- state_desc = ONLINE
- DBCC CHECKDB sin errores

Trampa: STOPAT va en HoraOK, antes del DELETE, y sin NORECOVERY en el último paso.

Caso práctico integrador final

15 min
RPO: log backup cada 15 minutos

60 min
RTO: restaurar y validar en una hora

3 roles
Mínimo privilegio: app, informes, DBA

1 / sem
DBCC CHECKDB semanal con alerta

Requisitos de Ventas S.A.

- **Seguridad:** logins y roles sin sysadmin.
- **Índices:** 2 justificados con su plan.
- **Backups:** Full, Diff y Log; prueba de restore.
- **Mantenimiento:** jobs, CHECKDB y alertas.

Evaluación y entregables

- **Peso:** 25 · 25 · 30 · 20 %.
- Script T-SQL comentado.
- Informe con RPO y RTO medidos.

RESUMEN

Checklist de buenas prácticas del DBA

Backups probados
Full + Diff + Log con CHECKSUM, copia externa y restore mensual.

Recovery Full en producción
Log backups cada 15 min según el RPO.

Integridad vigilada
PAGE_VERIFY CHECKSUM y CHECKDB semanal.

Mantenimiento por umbrales
Índices y estadísticas con scripts, no a ciegas.

Capacidad controlada
Autogrowth fijo, tempdb igual y espacio monitorizado.

Alertas operativas
Errores 823/824/825 y gravedad 19–25 por correo.

Mínimo privilegio
Roles ajustados y auditoría de sysadmin.

Runbook y DR
Procedimientos documentados y RPO/RTO medidos.

Gracias: ahora piensas como un DBA



Entiende el motor

Buffer Pool, WAL y VLFs explican rendimiento y recuperación.



Protege los datos

Mínimo privilegio y backups con CHECKSUM, probados.



Optimiza con datos

Índices, estadísticas y DMVs antes de tocar nada.



Automatiza y vigila

Jobs, alertas y runbooks: menos errores humanos.